Starlight Reinsurance System

A Comprehensive Guide to System Architecture, Actuarial Calculations, Data Pipelines, and Postman API Endpoint Specifications

DOCUMENT REF:	SRS-TECH-MANUAL-2026
VERSION:	1.5.1 (ITD Seeding Workflow Update)
DATE:	June 10, 2026
STATUS:	APPROVED / DEPLOYED
TARGET ENV:	PostgreSQL 15 / NestJS / Vue 3 SPA
AUTHOR:	Antigravity AI Assistant & Starlight Core Team

1. Executive Summary & Project Overview

The **Starlight Reinsurance System** is a secure enterprise ledgering platform designed to manage and calculate insurance treaty quota share premiums, ceding commissions, paid claims, loss reserves, and adjusting expenses across multiple underwriting states. It automatically generates reinsurance statement summaries, General Ledger (GL) account mappings, and Quota Share Cash Settlements from uploaded actuarial workbooks.

By implementing consecutive month-to-month chaining and automated data seeding, the system guarantees that reserve modifications (such as Case Reserves and IBNR) accurately carry over from month to month. Actuarial statements match Excel counterparts with 100% precision.

Core Business Problems Solved:

- **Manual Processing Elimination:** Automates the extraction of raw monthly premium and claim exhibits from complex workbooks.
- **Reserve Progression Tracking:** Ensures reserve adjustments (Case and IBNR) accurately chain from prior months, resolving baseline discrepancies.
- **Automated GL Entry Postings:** Maps statement outputs to standard double-entry accounting records (Debits & Credits) segmented by underwriting state.
- **Settlement Audit & Reconciliation:** Provides a clear view of cash balances due to/from reinsurers and third-party administrators (NTA).

2. System Architecture & Tech Stack

The system follows a modern decoupled **Client-Server Architecture** using a TypeScript-based stack. The database layer serves as a central relational store.

Layer	Technology Used	Key Role / Function
Frontend (Client SPA)	Vue 3, TypeScript, Vite, Vanilla CSS	Single Page Application (SPA) utilizing Composition API and custom CSS layouts. Features real-time overrides and premium light styling.
Backend (Server API)	NestJS, TypeORM, TypeScript	Enterprise-grade REST API managing workbooks, calculations, general ledger mappings, and seed execution.
Database	PostgreSQL (Port 5432)	Stores relational tables for workbooks, state-specific exhibits, and cash settlements.
Excel Parsing	xlsx (SheetJS) library	Extracts raw state exhibits and actuarial rates from uploaded Excel files.

3. Relational Database Schema & Structure

The PostgreSQL database contains the following key entities, mapped via TypeORM. Exhibits data is stored as 3-element numeric arrays representing **[Prior, Current, YTD]** values, which preserves the multi-dimensional structure of the reinsurance workbooks.

Entity: Workbook (`workbooks` table)

Field	Type	Description
id	Primary Key (Serial)	Unique identifier.
program	VARCHAR	Name of program, e.g., 'DPR APD', 'Excess NX'.
monthKey	VARCHAR	Date key formatted as YYYY-MM, e.g., '2026-01'.
monthLabel	VARCHAR	Formatted text, e.g., 'January 2026'.
source	VARCHAR	Identifies schema origin: 'ITD' (seeder), 'FUT' (workbook), or 'Starlight'.
rates	JSONB	Stores active rates: Quota Share, Ceding Comm %, ULAE %, Loss Pick %, DCC %, AOE %.

Entity: StateExhibit (`state_exhibits` table)

Field	Type	Actuarial Mapping / Meaning
workbookId	Foreign Key	Points to parent workbook record.
stateCode	VARCHAR(2)	State postal abbreviation or 'TOTAL'.
pw / pfw	NUMERIC[]	Direct written premium / Fronted written premium.
pc / pfc	NUMERIC[]	Direct commission / Fronted commission.
lp / laep / ae_paid	NUMERIC[]	Losses Paid / DCC Paid / AOE Paid.
uep / tax	NUMERIC[]	Unearned Premium Reserve / Premium Taxes.
lu / laeu / aeu	NUMERIC[]	Outstanding Case Reserves (Loss, DCC, AOE) from FUT.
loss_reserves	NUMERIC[]	Case Reserves - Loss (from ITD/Starlight).
loss_ibnr	NUMERIC[]	IBNR Reserves - Loss (from ITD/Starlight).
lae_reserves_dcc	NUMERIC[]	Case Reserves - DCC (from ITD/Starlight).
lae_ibnr_dcc	NUMERIC[]	IBNR Reserves - DCC (from ITD/Starlight).
lae_reserves_aoe	NUMERIC[]	Case Reserves - AOE (from ITD/Starlight).
lae_ibnr_aoe	NUMERIC[]	IBNR Reserves - AOE (from ITD/Starlight).
ulae_ibnr	NUMERIC[]	ULAE IBNR Reserves (from ITD/Starlight).

Entity: CashSettlement (`cash_settlements` table)

Field	Type	Description
id	Primary Key (Serial)	Unique identifier.
workbookId	Foreign Key	Points to parent workbook record (one-to-one relationship).
begBal	NUMERIC	Beginning cash balance on cash settlement statement.
amtPaid	NUMERIC	Amount paid/settled in current reporting period.

4. Core Actuarial & Reinsurance Calculations

The calculation engine is governed by strict insurance accounting regulations. Below are the core formulas implemented within the system:

Month-to-Month Chaining Flow

To evaluate change fields (e.g. Change in Loss Reserves), the system dynamically loads the previous month's ending reserves. January workbooks chain back to the **ITD (Inception-to-Date)** database seeds (representing December 2025 values). February workbooks chain back to January FUT, and so on.

1. Premiums Earned

```
Premiums Earned = Premiums Written + Change in UEP
Change in UEP = Prior Month UEP - Current Month UEP
```

2. Ultimate Loss & LAE (Loss Adjustment Expenses)

```
Ultimate Loss = Premiums Earned * Loss Pick %
Ultimate DCC = Premiums Earned * DCC %
Ultimate AOE = Premiums Earned * AOE %
```

3. Change in Reserves & Ending IBNR

```
Loss Reserves & IBNR:
    Change in Loss Reserves = Current Loss Reserves (lu) - Previous Loss Reserves
    Change in Loss IBNR = Ultimate Loss - Losses Paid - Change in Loss Reserves
    Current Loss IBNR = Previous Loss IBNR + Change in Loss IBNR
DCC Reserves & IBNR:
    Change in DCC Reserves = Current DCC Reserves (laeu) - Previous DCC Reserves
    Change in DCC IBNR = Ultimate DCC - DCC Paid - Change in DCC Reserves
    Current DCC IBNR = Previous DCC IBNR + Change in DCC IBNR
```

4. Unallocated Loss Adjustment Expense (ULAE)

```
ULAE Paid = Premiums Written * ULAE %
Change in ULAE IBNR = (0.5 * Change in Loss Reserves + Change in Loss IBNR) * 0.5%
Current ULAE IBNR = Previous ULAE IBNR + Change in ULAE IBNR
```

5. Net Settlement Formulas

```
A. Net Settlement due to/(from) Reinsurer:
    Net Settlement = Premiums Written - Ceding Commission - Losses Paid - DCC Paid - AOE Paid - ULAE Paid - Boards Charge - Loss Ratio Cap Charge
B. Net Settlement due from NTA:
    Net Settlement NTA = Premiums Written - Ceding Commission - Losses Paid - DCC Paid - AOE Paid - ULAE Paid
```

6. Required Collateral

```
Required Collateral = 115% * Total Loss & LAE Reserves
Total Loss & LAE Reserves = Loss Reserves + Loss IBNR + DCC Reserves + DCC IBNR + AOE Reserves + AOE IBNR + ULAE IBNR
```

Mutual Exclusivity of DCC & AOE Expenses & Reserves

To prevent double-counting and align with specific treaty terms, the ceding engine enforces strict mutual exclusivity between Defense and Cost Containment (DCC) and Adjusting & Other Expenses (AOE):

- **DCC Active Condition (laeDcc > 0%):** All AOE paid claims and AOE reserves are forced to zero in calculations. Any raw paid expenses are routed strictly to DCC.
- **AOE Active Condition (laeAoe > 0%):** All DCC paid claims and DCC reserves are forced to zero in calculations. Any raw paid expenses are routed strictly to AOE.
- **Both Inactive (laeDcc = 0% and laeAoe = 0%):** Both DCC and AOE paid claims and reserves are forced to zero.
- **FUT Paid Expense Isolation:** In ceding workbooks under the FUT schema, the row 21 field ae_paid represents ULAE-TPA fees (calculated as 7% of written premiums), not raw AOE paid claims. The calculation engine isolates this field and excludes it from DCC/AOE paid routing to prevent double ULAE/AOE deduction.

Verified Actuarial Calculation Example (CA State, Jan-26)

Actuarial Field	Calculated Value	Formula / Explanation
Premiums Written	**-45,686.42**	Direct raw input from January Excel row 14, col D (Phys. Damage)
Premiums Earned	**-1,087,262.86**	pw (-$45,686.42) + changeUEP (-$1,041,576.44)
Ultimate Loss	**-615,390.78**	premiumsEarned × 56.6% Loss Pick
Ending Loss IBNR	**-889,599.21**	prevLossIBNR (0) + changeLossIBNR (-$889,599.21)
Ending ULAE IBNR	**-3,951.12**	prevULAEIBNR (0) + changeULAEIBNR (-$3,951.12)

Verified Actuarial Calculation Example (CA State, Feb-26)

Actuarial Field	Calculated Value	Formula / Explanation
Premiums Written	**-21,355.30**	Direct raw input from February Excel row 14, col D (Phys. Damage)
Premiums Earned	**254,825.73**	pw (-$21,355.30) + changeUEP ($276,181.03)
Ultimate Loss	**144,231.36**	premiumsEarned × 56.6% Loss Pick
Change in Loss Reserves	**-4,230.75**	currLossRes ($194,518.13) - prevLossRes ($198,748.88)
Change in Loss IBNR	**-144,376.36**	ultimateLoss ($144,231.36) - paid ($292,838.47) - reservesChange
Ending Loss IBNR	**1,734,427.24**	prevLossIBNR ($1,878,803.60) + changeLossIBNR (-$144,376.36)
Ending ULAE IBNR	**5,158.43**	prevULAEIBNR ($5,890.89) + changeULAEIBNR (-$732.46)

5. Core System Data Flow & Lifecycle

The lifecycle of a workbook import and statement query follows these sequential steps:

1. **1. Seeding Base Data:** The seeder reads 38 state JSON files representing December 2025 ITD (Inception-to-Date) ending balances, aggregates them to create TOTAL.json, and saves the records as the baseline workbook.
2. **2. File Upload & Sheet Extraction:** The user uploads a monthly FUT Excel workbook via the UI. The NestJS backend parses it using xlsx, extracts the monthly column sheets (e.g. MTHLY-XX), maps rows 14 to 27, and persists state exhibits.
3. **3. Month Chaining Lookup:** When a statement is requested, the system searches the database for the prior month's workbook to establish the starting reserves. If January is requested, it chains to December ITD. If February is requested, it chains to January FUT.
4. **4. Statement Logic Calculation:** The backend calculates the changes in reserves, ULAE changes, and net settlements, returning the table rows to the Vue frontend.
5. **5. Frontend Grid Render & Parameters Override:** The Vue SPA renders the data in editable tables. If the user overrides rates (such as Loss Pick or Commission), the frontend recalculates all fields in real-time and permits saving the adjusted parameters.

System Data Lifecycle Flowchart

6. General Ledger (GL) Journal Entries Mapping

To synchronize reinsurance results with corporate finance, the system maps the statement calculations to a standard chart of accounts. Below are the key account codes, descriptions, and rules used in the mapping service:

Acct Code	Account Description	State Segment	Debit / Credit Rule	Calculation Basis
120100	Uncollected Premium Direct	00 (HQ)	Debit (Positive) / Credit (Negative)	Written Premium
400100	Direct Premium Written	State Code	Credit (Negative)	Written Premium
411100	Change in Unearned Prem Direct	State Code	Credit (Negative)	Change in UEP
230100	Unearned Premium Direct	State Code	Debit (Positive)	Change in UEP
400300	Ceded Premium Written	00 (HQ)	Debit (Positive)	Written Premium
243100	Ceded Reinsurance Premium Payable	00 (HQ)	Credit (Negative)	Written Premium
230300	Unearned Premium Ceded	00 (HQ)	Credit (Negative)	Change in UEP
411300	Change in Unearned Prem Ceded	00 (HQ)	Debit (Positive)	Change in UEP
500100	Commissions Direct	State Code	Debit (Positive)	Ceding Commission
120100	Uncollected Premium Direct	00 (HQ)	Credit (Negative)	Ceding Commission
243100	Ceded Reinsurance Premium Payable	00 (HQ)	Debit (Positive)	Ceding Commission
500300	Commissions Ceded	State Code	Credit (Negative)	Ceding Commission

7. Parameter Routing & Robustness Enhancements

Several sophisticated mechanisms have been built into the system to handle actuarial exceptions and optimize frontend rendering performance.

DCC to AOE Parameter Fallback (DPR APD Program)

For programs like DPR APD, Defense and Cost Containment (DCC) rates are set to 0.00%, meaning all adjusting expense liabilities reside under Adjusting & Other Expense (AOE). However, the general user form template only exposes a parameter labeled *Previous LAE IBNR Reserves - DCC*.

To prevent database contamination and UI confusion, the frontend composable useJournalEntries.ts implements smart fallback routing logic:

- **On Load:** If DCC IBNR is 0, the input field automatically maps to and displays the value of lae_ibnr_aoe (AOE).
- **On Save:** When the user saves modified parameters, if the program's DCC pick rate or DCC reserves are 0, the backend routes the override value directly to the lae_ibnr_aoe database column instead of lae_ibnr_dcc.
- **Result:** The database columns remain 100% accurate, while the user has a seamless single-input experience.

Sequential reserves chaining correction for FUT workbooks

Historically, ceding statements generated for FUT workbooks reset the Previous ULAE, Loss, and DCC IBNR parameters to 0. This occurred because FUT Excel workbooks do not contain detailed reserves on upload, causing the system to fall back to the raw unpaid columns (aeu, lu, laeu) which were blank (0) in the uploaded monthly FUT sheet.

Solution: We modified the logic in useJournalEntries.ts and reports.service.ts to check if the prior FUT workbook has calculated detailed reserves in the database. If so (using array .some(v => Number(v) !== 0) verification), the system loads the prior month's ending reserves directly from the calculated database fields (loss_ibnr, ulae_ibnr, lae_ibnr_aoe) rather than falling back to the raw unpaid columns. This allows reserves to flow sequentially month-over-month.

Chrome Backdrop-Filter rendering bug

During testing, we identified that applying the CSS class .card-panel (which uses backdrop-filter for glassmorphic aesthetics) alongside overflow-x: auto on the same HTML wrapper caused Google Chrome's compositor to drop painted child nodes, resulting in a blank white box overlaying the table grids.

Solution: We separated these concerns into parent-child nested DOM structures in ReinsuranceStatementTable.vue, GLJournalEntriesTable.vue, and StateExhibitGrid.vue:

```
<div class="card-panel">
    <div class="table-container" style="overflow-x: auto;">
        <table>...</table>
    </div>
</div>
```

Sequential Month Validation & Overwrite Confirmation

To prevent invalid calculations due to gaps in reporting data (e.g. attempting to chain February ceding values directly to Inception-to-Date base seeds without January in the database), the backend and frontend enforce sequential checks:

- **Sequential Month Enforcer:** When importing a month (e.g., YYYY-MM), the system automatically validates that the previous month's workbook (e.g., YYYY-MM minus 1) already exists in the database. Gaps in reporting month sequence are rejected with a 400 Bad Request validation error. For January, the system checks that the base December 2025 ITD workbook is seeded first.
- **Interactive Overwrite Confirmation:** When a file is uploaded, if a workbook for the same program and month already exists, the API rejects it with a 409 Conflict response rather than silently overwriting. The frontend catches this error and displays an interactive prompt asking the user: *'Do you want to overwrite it and store the same file data?'*. If the user confirms, the request is re-submitted with overwrite=true to safely replace the record.

ITD Excel Generator Feature

To allow actuarial teams to quickly seed new programs or reset baselines, the system features a built-in ITD Excel Generator. When a user uploads a complete Starlight summary sheet (Southlake workbook), the system:

- Parses the aggregate Column C (representing *Totals* / *ITD* values) for all state sheets.
- Formulates lu (Total Loss Reserves = loss_reserves + loss_ibnr), laeu (Total DCC Reserves = lae_reserves_dcc + lae_ibnr_dcc), and aeu (Total AOE Reserves = lae_reserves_aoe + lae_ibnr_aoe + ulae_ibnr).
- Generates a new ceding-formatted workbook where each sheet has the name MTHLY-XX (plus MTHLY-TOTAL).
- Sets cell C4 to Dec-25 (December 2025 baseline) and writes out the ITD values into rows 14 to 34.
- Allows the user to download this sheet and upload it directly as a seeder workbook. The backend automatically detects the seeder filename/date and writes it directly to the database with source ITD.

8. API Documentation (REST & Postman Specification)

The backend is structured as a RESTful web service. Integrating applications, scripting agents, or Postman collections can interact with the ledger using the following specifications. All requests assume a content type of application/json unless stated otherwise.

Base Configuration

Base URL: http://localhost:3000
Development Headers:
Content-Type: application/json

Method	Endpoint Path	Function / Role	Payload Details
POST	/api/database/clear	Reset all database tables	None
POST	/api/database/seed	Seeds baseline Dec 2025 ITD workbook	None
GET	/api/database/check-itd-seeded	Check seeding status	Returns { seeded: boolean }
GET	/api/workbooks	List all workbooks in DB	Returns array of workbook objects
GET	/api/workbooks/:id	Get workbook and exhibits by ID	Returns detailed workbook tree
DELETE	/api/workbooks/:id	Remove workbook from database	HTTP 204 No Content
POST	/api/workbooks/upload	Upload Excel workbook file	multipart/form-data with key 'file'
POST	/api/workbooks/generate-itd	Generate ITD seeder Excel workbook	multipart/form-data with key 'file' (Starlight summary file)
PUT	/api/workbooks/:id/rates	Override actuarial rates	JSON rate mapping parameters
PUT	/api/workbooks/:id/exhibits/:stateCode	Override state exhibit reserves	JSON reserve array mapping
GET	/api/workbooks/:id/reinsurance-statement/:stateCode	Calculate Quota Share statement	State exhibit calculations
GET	/api/workbooks/:id/gl-journal-entries/:stateCode	Retrieve mapped GL debits/credits	State-level journal entry mappings
GET	/api/workbooks/:id/cash-settlement-calculations	Retrieve cash settlement rollup	System-wide aggregates

Detailed Endpoint Payloads & Responses

1. Upload Excel Workbook

Endpoint: POST /api/workbooks/upload
Content-Type: multipart/form-data
Payload: Form field file binary upload.
Sample Response (JSON):


```
{ "id": 12, "program": "DPR APD", "monthKey": "2026-01", "monthLabel": "January 2026", "source": "FUT",
"rates": { "qs": 100, "cf": 5, "comm": 32, "ulae": 1, "xol": 2, "lossPick": 56.6, "laeDcc": 0, "laeAoe":
13.4, "boardsCharge": 0.4 }, "createdAt": "2026-06-09T12:00:00.000Z" }
```


2. Override Actuarial Rates

Endpoint: PUT /api/workbooks/:id/rates
Sample Payload (JSON):


```
{ "comm": 32.5, "lossPick": 56.6, "laeDcc": 0.0, "laeAoe": 13.4 }
```


Sample Response (JSON): Returns the updated workbook entity schema.

3. Get Quota Share Reinsurance Statement

Endpoint: GET /api/workbooks/:id/reinsurance-statement/:stateCode
Sample Response (JSON Array):


```
[ { "label": "Premiums Written", "value": 1485612.05, "isBold": true }, { "label": "Change in UEP",
"value": -845612.05 }, { "label": "Premiums Earned", "value": 640000.0, "isBold": true, "borderClass":
"single-underline" }, { "label": "Net Settlement due to/(from) Reinsurer", "value": -13748.88, "isBold":
true }, { "label": "Net Settlement due from NTA", "value": -13797.54, "isBold": true } ]
```


4. Get Mapped GL Journal Entries

Endpoint: GET /api/workbooks/:id/gl-journal-entries/:stateCode
Sample Response (JSON Array):


```
[ { "desc": "Uncollected Premium Direct", "comp": "100", "account": "120100", "cc": "000", "mga": "1201",
"lob": "000171", "st": "CA", "debit": 1485612.05, "credit": 0 }, { "desc": "Direct Premium Written",
"comp": "100", "account": "400100", "cc": "000", "mga": "1201", "lob": "000171", "st": "CA", "debit": 0,
"credit": 1485612.05 } ]
```